

// Jivops

Guía completa — 30 lecciones

Arquitectura · Sources · Reconciliación · Multi-tenancy y estructura ·
Automatización de imágenes · Operación y madurez

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Flux y el modelo GitOps Toolkit
- Lección 02 — Arquitectura — controllers especializados (source, kustomize, helm, notification)
- Lección 03 — Instalación de Flux (flux bootstrap)
- Lección 04 — Flux CLI — comandos fundamentales
- Lección 05 — Flux vs ArgoCD — diferencias de enfoque

Módulo 2: Sources

- Lección 06 — GitRepository — fuente desde Git
- Lección 07 — HelmRepository — fuente desde repositorios Helm
- Lección 08 — OCIRepository — fuente desde registries OCI
- Lección 09 — Bucket — fuente desde object storage
- Lección 10 — Autenticación de sources (SSH, tokens, credenciales)

Módulo 3: Reconciliación de recursos

- Lección 11 — Kustomization CRD — aplicar manifiestos desde un source
- Lección 12 — HelmRelease — gestión de Helm charts
- Lección 13 — Dependencias entre Kustomizations
- Lección 14 — Intervalos de reconciliación y drift detection
- Lección 15 — Health checks y readiness en Kustomizations

Módulo 4: Multi-tenancy y estructura de repos

- Lección 16 — Patrón de repositorios — monorepo vs multi-repo
- Lección 17 — Estructura recomendada (clusters/, infrastructure/, apps/)
- Lección 18 — Multi-tenancy con Flux — tenants aislados
- Lección 19 — Promoción entre entornos (dev → staging → prod)
- Lección 20 — Flux y Kustomize — integración nativa

Módulo 5: Automatización de imágenes y notificaciones

- Lección 21 — Image Reflector Controller — escaneo de nuevas versiones
- Lección 22 — Image Automation Controller — actualización automática de tags
- Lección 23 — Políticas de selección de imágenes (semver, alfabético)
- Lección 24 — Notification Controller — alertas hacia Slack/Teams
- Lección 25 — Webhooks — receivers para triggers externos

Módulo 6: Operación y madurez

Lección 26 — RBAC y seguridad en Flux

Lección 27 — Troubleshooting de reconciliación fallida

Lección 28 — Monitoreo de Flux con Prometheus/Grafana

Lección 29 — Actualización de Flux sin downtime

Lección 30 — Proyecto final: plataforma GitOps completa con Flux lista para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Flux y el modelo GitOps Toolkit

Cada controller especializado puede escalarse o actualizarse según su propia carga, sin coordinar cambios en un único componente monolítico.

Lección 02 — Arquitectura — controllers especializados (source, kustomize, helm, notification)

El source-controller sincroniza una fuente una sola vez; múltiples Kustomizations reutilizan ese contenido sin descargarlo por separado cada una.

Lección 03 — Instalación de Flux (flux bootstrap)

Flux escribe su propia configuración en el repositorio Git gestionado, así que también se administra a sí mismo bajo el mismo principio de GitOps.

```
flux bootstrap github --owner=miorg --repository=mi-repo
```

Lección 04 — Flux CLI — comandos fundamentales

'flux reconcile' fuerza una sincronización inmediata, acelerando el ciclo de feedback frente a esperar el intervalo programado normal.

```
flux reconcile kustomization mi-app
```

Lección 05 — Flux vs ArgoCD — diferencias de enfoque

Flux se integra profundamente con CRDs nativos de Kubernetes, resultando más natural para equipos que prefieren gestionar todo vía kubectl.

Módulo 2: Sources

Lección 06 — GitRepository — fuente desde Git

Definir el GitRepository como recurso declarativo permite versionar y auditar la propia configuración de fuentes igual que el resto del cluster.

```
url: https://github.com/miorg/mirepo
```

Lección 07 — HelmRepository — fuente desde repositorios Helm

Múltiples HelmReleases pueden reutilizar el mismo HelmRepository ya declarado, sin repetir la configuración de conexión en cada uno.

```
url: https://charts.miorg.com
```

Lección 08 — OCIRepository — fuente desde registries OCI

Reutiliza infraestructura de seguridad, firma y distribución ya establecida para imágenes, aplicándola también a la distribución de manifiestos.

Lección 09 — Bucket — fuente desde object storage

Un proceso automatizado que genera configuración dinámicamente puede preferir publicar a un bucket sin implementar un flujo de commit Git formal.

```
bucketName: mis-manifiestos
```

Lección 10 — Autenticación de sources (SSH, tokens, credenciales)

'secretRef' mantiene la credencial separada del manifiesto versionado, evitando que quede expuesta en el historial de Git.

```
secretRef: name: mi-credencial-git
```

Módulo 3: Reconciliación de recursos

Lección 11 — Kustomization CRD — aplicar manifiestos desde un source

El kustomize-controller tiene la lógica de Kustomize integrada internamente, sin depender de un binario externo instalado por separado.

```
path: ./overlays/production
```

Lección 12 — HelmRelease — gestión de Helm charts

La reconciliación continua detecta y revierte un cambio manual hecho fuera de Git, manteniendo esa única fuente de verdad consistente.

```
chart: mi-chart
```

Lección 13 — Dependencias entre Kustomizations

Sin dependsOn declarado, una Kustomization podría fallar al intentar usar una CRD que otra Kustomization todavía no ha instalado.

```
dependsOn: - name: infraestructura-base
```

Lección 14 — Intervalos de reconciliación y drift detection

Un intervalo muy corto genera carga innecesaria; uno muy largo retrasa detectar un drift real entre Git y el cluster.

```
interval: "5m"
```

Lección 15 — Health checks y readiness en Kustomizations

Sin health check, una Kustomization dependiente podría empezar prematuramente asumiendo que algo ya está listo cuando en realidad no lo está.

```
kind: Deployment
```

Módulo 4: Multi-tenancy y estructura de repos

Lección 16 — Patrón de repositorios — monorepo vs multi-repo

Un equipo único se beneficia de coordinar todo en un solo lugar; una organización con muchos equipos se beneficia más del aislamiento de multi-repo.

Lección 17 — Estructura recomendada (clusters/, infrastructure/, apps/)

Separar infrastructure/ de apps/ facilita aplicar distintos niveles de revisión según el impacto y criticidad del cambio.

```
clusters/produccion/
```

Lección 18 — Multi-tenancy con Flux — tenants aislados

Un service account distinto por tenant con RBAC limitado restringe técnicamente qué puede tocar cada equipo, más allá de una separación solo organizativa.

```
serviceAccountName: tenant-a-sa
```

Lección 19 — Promoción entre entornos (dev → staging → prod)

El pull request de promoción da un punto de control humano antes de que un cambio llegue a un entorno más crítico como producción.

Lección 20 — Flux y Kustomize — integración nativa

Integrar la lógica de Kustomize dentro del propio controller elimina una dependencia externa adicional que gestionar por separado.

Módulo 5: Automatización de imágenes y notificaciones

Lección 21 — Image Reflector Controller — escaneo de nuevas versiones

Separar 'descubrir tags' de 'decidir cuál usar' permite cambiar el criterio de selección sin tocar la lógica de escaneo del registry.

```
kind: ImageRepository
```

Lección 22 — Image Automation Controller — actualización automática de tags

El controller hace un commit en Git antes de desplegar, manteniendo a Git como la única fuente de verdad del principio GitOps.

Lección 23 — Políticas de selección de imágenes (semver, alfabético)

Restringir el rango semver excluyendo versiones mayores evita que la automatización salte a un breaking change sin ninguna revisión previa.

```
range: ">=1.0.0 <2.0.0"
```

Lección 24 — Notification Controller — alertas hacia Slack/Teams

Separar Provider (destino) de Alert (evento) permite reutilizar el mismo canal de Slack en múltiples Alerts sin repetir la configuración.

```
kind: Provider
```

Lección 25 — Webhooks — receivers para triggers externos

Un Receiver reacciona de inmediato al push, acortando significativamente el tiempo de espera frente a depender solo del intervalo programado.

```
kind: Receiver
```


Módulo 6: Operación y madurez

Lección 26 — RBAC y seguridad en Flux

Un service account con permisos mínimos limita el alcance del daño posible si la configuración de Flux tuviera algún problema de seguridad.

Lección 27 — Troubleshooting de reconciliación fallida

El status de la Kustomization específica da un mensaje directo del problema, más útil que revisar logs generales mezclados de todo el controller.

Lección 28 — Monitoreo de Flux con Prometheus/Grafana

Una alerta sobre el tiempo desde la última reconciliación exitosa detecta proactivamente un estancamiento silencioso antes de que alguien lo note manualmente.

Lección 29 — Actualización de Flux sin downtime

Un cambio de API en los CRDs podría romper manifiestos existentes; detectarlo en staging evita romper reconciliaciones masivas en producción.

Lección 30 — Proyecto final: plataforma GitOps completa con Flux lista para producción

Se integra todo lo anterior: estructura de repos clara, Kustomizations/HelmReleases con dependencias correctas, multi-tenancy con RBAC, automatización de imágenes con políticas seguras, notificaciones configuradas y monitoreo con Prometheus — el nivel esperado de una plataforma GitOps con Flux lista para producción real.